SWIGLU AS A LOW-RANK TENSOR MODEL

ANIKET DESHPANDE

In this brief note, I explore a tensor-theoretic interpretation of the transformer feedforward network (FFN), viewing the SwiGLU block as a highly structured low-rank interaction layer over the residual stream. I show that SwiGLU admits an exact input-dependent second-order interaction tensor with canonical polyadic (CP) factorization, together with a diagonal pairing of latent features in hidden space. This in turn suggests a principled generalization via a learned low-rank Tucker core that allows the model to learn which latent features should interact multiplicatively.

note: this idea is in *very* early stages. I'm actively working on more concrete theory and experiment design and will update the note as I make progress. if you have access to this note and wish to discuss ideas/fixes/implementation, please reach out! email: aniket4@illinois.edu

I. Introduction

We begin with the residual stream. At a fixed token position, let $x \in \mathbb{R}^d$ denote the residual stream vector entering the feedforward network (FFN) block. In a modern transformer [Vas+17], attention is responsible for moving information across tokens, while the FFN acts pointwise on each token's residual representation. In the architectures of relevance, this pointwise map is usually a gated MLP of the SwiGLU form [Sha20]

$$y = U^T h, \quad h = \left(W^T x\right) \odot \mathrm{SiLU}\left(G^T x\right), \tag{1}$$

where $W, G \in \mathbb{R}^{d\times m}$ and $U \in \mathbb{R}^{m\times d}$. Here, W is the main up-projection, G is the gate projection, and U is the down-projection, and $h \in \mathbb{R}^m$ is the hidden activation. Writing the j-th columns of W and G as $w_j, g_j \in \mathbb{R}^d$, the j-th hidden coordinate is

$$h_j(x) = \left(w_j^\top x\right) \mathrm{SiLU}\left(g_j^\top x\right). \tag{2}$$

Already, we can see a core structural difference between the FFN and the attention map. From the kernel-regression viewpoint, it is Nadaraya-Watson regression with a Gaussian kernel [Nad64, Pai25]. From a statistical mechanical viewpoint, the attention weights form a Boltzmann distribution over key-value pairs,

$$\alpha_i(q) = \frac{\exp\left(\beta q^T k_i\right)}{\sum_j \exp\left(\beta q^T k_j\right)},$$

with inverse temperature $\beta = \tau^{-1}$. The same mechanism therefore interpolates continuously between diffuse averaging at high temperature and sharp retrieval at low temperature. This dual interpretation is part of attention's tractability. The FFN, specifically the SwiGLU architecture, has no comparable theory. Yet it contains the majority of the model's parameters and is plausibly where many of the remaining gains could lie. I'd like to analyze the FFN from a tensor perspective.

II. Bilinear Structure of SwiGLU

Going back to (2), each unit computes a two-feature interaction. It is important to ask, what *kind* of interaction is it, and what structural assumptions does the layer impose on the space of possible feature interactions?

For SwiGLU, the key point is that the nonlinearity already has an exact multiplicative factorization. Since

$$\mathrm{SiLU}(z) = z\, s(z), \quad s(z) = \frac{1}{1+e^{-z}}, \tag{3}$$

we may rewrite the j-th hidden unit exactly as

$$h_j(x) = (w_j^\top x)\mathrm{SiLU}(g_j^\top x) = (w_j^\top x)(g_j^\top x)s(g_j^\top x). \tag{4}$$

Thus each hidden unit is an exact bilinear interaction between two learned scalar projections, modulated by an input-dependent scalar gate. The quadratic part is therefore an exact structural component of the unit for every input.

The factor

$$(w_j^\top x)(g_j^\top x) = x^\top (w_j g_j^\top) x$$

has a clean algebraic interpretation. Because $w_j g_j^\top$ is an outer product, it is a matrix of rank one. So each hidden unit builds a multiplicative interaction out of a rank-one separable interaction between two learned directions in residual space, together with a scalar routing coefficient $s(g_j^\top x)$. Each hidden unit sees two directions in residual space, projects onto them, then multiplies those two scalar responses, and then modulates the strength of that interaction by a gate. This is a very specific inductive bias. The model is not learning an arbitrary interaction pattern among latent features; it is learning a diagonal pairing of two projected feature sets.

III. CP Decomposition of the Interaction Tensor

The tensor viewpoint makes this more explicit. Define the quadratic feature tensor

$$x \otimes x \in \mathbb{R}^{d\times d}, \quad (x\otimes x)_{ab} = x_a x_b.$$

A fully general second-order hidden layer with m interaction channels would require a third-order tensor $\mathcal{T} \in \mathbb{R}^{m\times d\times d}$ with hidden coordinates

$$h_j(x) = \sum_{a=1}^{d}\sum_{b=1}^{d} \mathcal{T}_{jab} x_a x_b = \langle \mathcal{T}_j, x\otimes x\rangle. \tag{5}$$

However, SwiGLU does not allow arbitrary slices $\mathcal{T}_{j,:,:}$. Instead, by (4)

$$h_j(x) = s(g_j^\top x)\langle w_j \otimes g_j, x\otimes x\rangle = s(g_j^\top x)(w_j^\top x)(g_j^\top x). \tag{6}$$

So every hidden slice is forced to factor as an outer product, and the nonlinearity acts only through an input-dependent scalar coefficient. Now, we include the output projection. The output becomes

$$y_o = \sum_{j=1}^{m} U_{jo} h_j(x) = \sum_{j=1}^{m} U_{jo} s(g_j^\top x)(w_j^\top x)(g_j^\top x) \tag{7}$$

$$= \sum_{a=1}^{d}\sum_{b=1}^{d}\left(\sum_{j=1}^{m} U_{jo} s(g_j^\top x) w_{ja} g_{jb}\right) x_a x_b. \tag{8}$$

We therefore define the exact input-dependent interaction tensor $\mathcal{A} \in \mathbb{R}^{d\times d\times d}$

$$y_o = \sum_{a,b} \mathcal{A}_{oab}(x) x_a x_b, \quad \mathcal{A}_{oab}(x) = \sum_{j=1}^{m} U_{jo} s(g_j^\top x) W_{aj} G_{bj}. \tag{9}$$

This is precisely a canonical polyadic (CP) decomposition [KB09] of the interaction tensor

$$\mathcal{A}(x) = \Big[\Big[U^\top \mathrm{diag}(\alpha(x)), W, G\Big]\Big] = \sum_{j=1}^{m} \alpha_j(x) u_j \otimes w_j \otimes g_j, \quad \alpha_j(x) = s(g_j^\top x), \tag{10}$$

where u_j is the output factor associated with the j-th hidden unit. So, for every input x, SwiGLU realizes an exact input-routed family of low-rank CP interaction tensors.

SwiGLU realizes multiplicative feature mixing [JSSB20], but only through sums of rank-one separable factors. Even more strongly, the routing is coordinate-wise in the hidden space: the j-th feature in one branch is only allowed to interact with the j-th feature in the other branch, and the resulting rank-one interaction atom $u_j \otimes w_j \otimes g_j$ is gated only by the scalar sigmoid $s(g_j^\top x)$ associated with that channel. This is a useful bias; it gives multiplicative structure at low cost. However, it is also a strong restriction. The model is not learning an arbitrary interaction pattern among latent features; it is learning a diagonal pairing of two projected feature sets, where only same-index latent features are allowed to interact. In tensor speak, SwiGLU imposes a superdiagonal interaction core in latent space.

IV. Tucker Generalization

This naturally suggests a more principled generalization. Instead of forcing the interaction routing to be same-index in hidden space, we introduce a learned low-rank core tensor. The simplest way to do this is through a Tucker decomposition. We choose factor matrices $P, Q \in \mathbb{R}^{d\times r}$ and $R \in \mathbb{R}^{d\times s}$, and a core tensor $\mathcal{C} \in \mathbb{R}^{s\times r\times r}$. We project the residual stream into two latent feature spaces $p = P^T x \in \mathbb{R}^r$ and $q = Q^T x \in \mathbb{R}^r$, then define interaction channels as

$$z_\alpha = \sum_{i=1}^{r}\sum_{j=1}^{r} \mathcal{C}_{\alpha ij} p_i \mathrm{SiLU}(q_j), \quad \alpha = 1, \ldots, s, \tag{11}$$

and map them back to residual space with $y = Rz$. This is a Tucker-factorized generalization of SwiGLU. In SwiGLU, each projected feature p_i can only interact with the corresponding gated feature q_i. In the Tucker formalism, the learned core $\mathcal{C}$ decides which projected features should interact multiplicatively. Same-index interactions become a special case when $\mathcal{C}$ is superdiagonal.

Using again the identity $\mathrm{SiLU}(z) = zs(z)$, we can equivalently write the interaction channels as

$$z_\alpha = \sum_{i=1}^{r}\sum_{j=1}^{r} \mathcal{C}_{\alpha ij} s(q_j) p_i q_j, \tag{12}$$

so this generalized block still realizes an input-dependent interaction tensor, but now with a learned low-rank cross-channel core rather than same-index routing only. This generalization allows the model to learn richer interaction patterns among latent features, while still maintaining a low-rank structure that keeps the parameter count manageable.

We can directly recover SwiGLU if $r = s = m$ and $\mathcal{C}_{\alpha ij} = \delta_{\alpha i}\delta_{ij}$. Then, $z_i = p_i \mathrm{SiLU}(q_i)$, which is exactly the original elementwise gating pattern. So Tucker strictly generalizes SwiGLU rather than replacing it with an unrelated construction.

This direction is promising for three main reasons. First, it is driven by a principled analysis that characterizes the exact interaction structure of the FFN, rather than a local approximation whose validity depends on the operating regime. Second, it introduces a clean tradeoff between expressivity and compute via r and s. The latent rank r and the number of interaction channels s control how much multiplicative structure the block can represent. Third, the contraction should remain reasonably hardware-friendly, since it can be implemented through batched matrix multiplications and small tensor contractions rather than an unstructured dense higher-order tensor.

V. Interpretability

There is also an interesting interpretability motivation. One reason attention has been unusually amenable to mechanistic analysis, and why such analysis has already enabled principled architecture design [Ols+22, Fu+23], is that its structure is additive and probabilistic: attention heads decompose into weights over values, and those weights often admit clear semantic or algorithmic roles. The FFN block has not yielded to the same kind of decomposition. One plausible reason is that its interaction geometry is much harder to factor cleanly downstream. But the exact representation above suggests that SwiGLU already decomposes into a dictionary of fixed rank-one interaction atoms $u_j \otimes w_j \otimes g_j$ together with input-dependent routing coefficients $\alpha_j(x) = s(g_j^\top x)$. This separates what interaction is available from when that interaction is used.

If the multiplicative structure inside the FFN were made more explicit and low-rank, then sparse decomposition methods (dictionary methods, sparse autoencoders, transcoders) [Bri+23] might have a cleaner object to work with. Rather than trying to recover structure only from downstream activations, one could try to analyze the routed interaction channels themselves:

$$c_j(x) = \alpha_j(x)(w_j^\top x)(g_j^\top x).$$

From a rate-distortion perspective [PKW24], current FFNs may be hiding structure in a way that makes it difficult to disentangle without a more explicit inductive bias. By contrast, the proposed Tucker generalization would encourage the model to learn a more explicit interaction structure, which could in turn facilitate mechanistic interpretability without sacrificing the empirical advantage of multiplicative gating.

VI. Research Updates

April 1, 2026. Before working on anything novel, I wanted to do an empirical characterization of SwiGLU's routed-CP structure. Specifically, I wanted to check whether the CP decomposition of SwiGLU, which is algebraically exact, reveals meaningful structure in pretrained models. The mathematics guarantees a certain factorization, but does not guarantee that the factorization is informative or meaningful. If the routing coefficients α_i are all approximately constant, or if channel contributions were uniformly spread, then the decomposition would be correct but inert. These experiments characterize how a trained model actually uses the structure the decomposition exposes.

All experiments use Qwen2.5-0.5B (24 SwiGLU layers, hidden dimension $d = 896$, intermediate dimension $m = 4864$), running inference on a 4096-token chunk of WikiText-2.

Recall the SwiGLU forward pass. At a fixed token position, let $x \in \mathbb{R}^d$ be the residual stream vector entering the FFN. The layer computes

$$y = U^\top h, \quad h_j(x) = (w_j^\top x) \cdot \mathrm{SiLU}(g_j^\top x),$$

where $W, G \in \mathbb{R}^{d\times m}$ are the up-projection and gate projection, and $U \in \mathbb{R}^{m\times d}$ is the down-projection. Using the identity $\mathrm{SiLU}(z) = zs(z)$, we can rewrite the hidden units as

$$h_j(x) = \underbrace{(w_j^\top x)(g_j^\top x)}_{\textit{bilinear interaction}} \cdot \underbrace{s(g_j^\top x)}_{\alpha_j(x)}.$$

The first term is a fixed quadratic form, a rank-one separable interaction between two learned directions in residual space. The second term, $\alpha_j(x) = s(g_j^\top x) \in [0, 1]$, is an input-dependent scalar gate that routes the interaction: for each input x, it decides how strongly channel j's interaction contributes to the output.

Let us define the channel contribution $c_j(x) = \alpha_j(x)(w_j^\top x)(g_j^\top x)$, so that the output can be written as $y = \sum_j u_j c_j(x)$, and the layer realizes an exact input-dependent CP decomposition of a third-order interaction tensor

$$\mathcal{A}(x) = \sum_{j=1}^{m} \alpha_j(x) u_j \otimes w_j \otimes g_j.$$

This separates the layer's computation into two parts: the fixed rank-one atoms $u_j \otimes w_j \otimes g_j$, which define what interactions are available, and the input-dependent routing coefficients $\alpha_j(x)$, which define when those interactions are used. The CP decomposition exposes this structure explicitly. The question is, does the model actually use it? We probe the following in this set of experiments:

(1) **Routing coefficient statistics**. We examine the behavior of α_j across tokens and layers. This characterizes the input-dependent part of the decomposition: are routing coefficients variable enough to do real input-dependent selection, or are they roughly constant (reducing SwiGLU to a static bilinear layer)?
(2) **Channel sparsity**. We examine the distribution of channel contributions $|c_j|$ per token. This asks whether the routed-CP decomposition is effectively low-rank on real inputs: whether the routing selects a small active subset of atoms per token, or whether contributions are more diffusely spread across m channels.
(3) **Routing ablation**. We replace the input-dependent routing $\alpha_j(x)$ with constants and measure the effect on model performance. This is a counterfactual test: how much of the model's computation depends on input-dependent routing versus the static bilinear interaction structure?

VI.i. **Routing Coefficient Statistics**. For each SwiGLU layer ℓ and each channel $j \in \{1, \ldots, m\}$, we compute $\alpha_j(x) = s(g_j^\top x)$ for every token x in the evaluation corpus. We then compute two summary statistics:

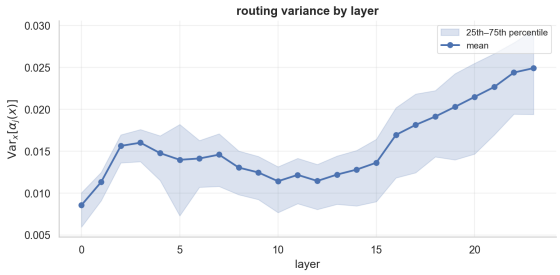
- Per-channel variance $s_j = \mathrm{Var}_x[\alpha_j(x)]$. A channel with $s_j \approx 0$ has a roughly constant routing coefficient across all inputs: it is effectively a static bilinear atom, and the sigmoid gate is not performing input-dependent selection. A channel with $s_j > 0$ has a variable routing coefficient, so it is performing some degree of input-dependent selection.
- Marginal distribution of α values across all channels and tokens within a layer. This reveals where gate pre-activations typically sit on a sigmoid curve.

The α distribution shifts and sharpens with depth (Figure 2). In layer 0, the distribution is roughly symmetric and centered near 0.5, meaning gate pre-activations $g_j^\top x$ are typically near zero, where σ is approximately linear. Early layers operate in a regime where routing is soft: every channel receives moderate activation, and the sigmoid acts more as a smooth rescaling than a selection mechanism.

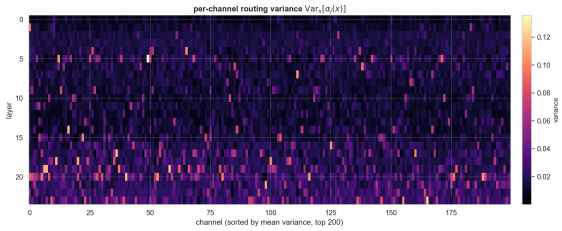
By layers 8 and 16, the distribution shifts leftward, with the mode around 0.25–0.35. Most channels are being partially suppressed most of the time. The gate pre-activations have become more negative on average, pushing sigmoid outputs toward the flat region near zero.

Layer 23 develops bimodal structure. A bulk of channels have low α (the leftward-shifted main mass) plus a sharp spike at $\alpha \approx 1.0$. This spike represents channels whose gate pre-activations are so large and positive that the sigmoid saturates—these channels are always on, regardless of input. They have become permanent bilinear features. The final layer has learned a mixed strategy: a subset of channels performs fixed, input-invariant computation, while the remainder routes selectively.

Routing variance increases in later layers (Figure 1a). Mean per-channel variance follows a U-shaped curve across depth, with a minimum around layers 10–12 and a steep rise toward the final layers. Later layers have the most input-dependent routing—they perform the most conditional computation in the routed-CP sense. The heatmap (Figure 1b) shows that this high variance is



(A) Mean per-channel routing variance $\mathrm{Var}_x[\alpha_j(x)]$ averaged over channels, by layer. Variance follows a U-shaped curve across depth, with a minimum around layers 10–12 and a steep rise toward the final layers.



(B) Heatmap of per-channel routing variance across layers. High variance is concentrated in a sparse subset of channels rather than spread uniformly; most channels have low routing variance, and a small minority carry the bulk of input-dependent selection.

FIGURE 1. Routing variance statistics for Qwen2.5-0.5B across all 24 SwiGLU layers, evaluated on a 4096-token chunk of WikiText-2.

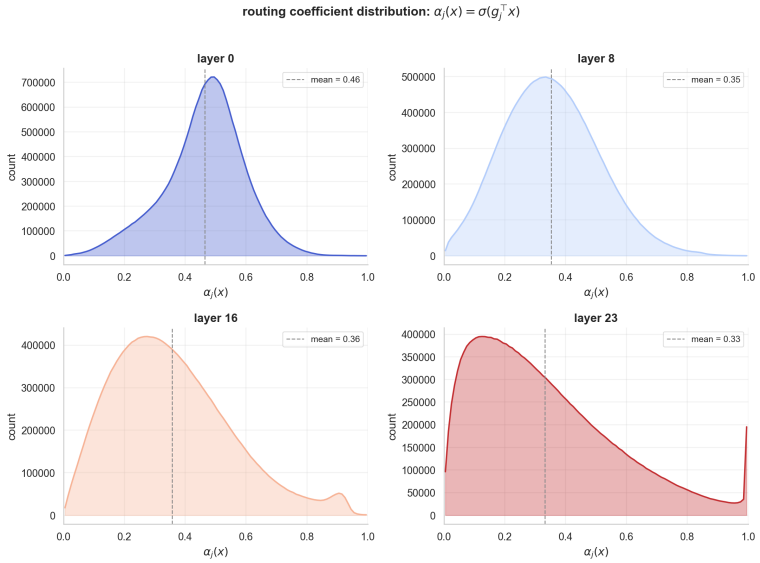


FIGURE 2. Marginal distribution of routing coefficients $\alpha_j(x) = s(g_j^\top x)$ across all channels and tokens, shown for selected layers. Early layers exhibit a roughly symmetric distribution centered near 0.5; deeper layers shift leftward and develop bimodal structure, with a spike at $\alpha \approx 1$ in the final layer.

concentrated in a sparse subset of channels, not spread uniformly. Most channels across the network have low routing variance; a small minority carry the bulk of input-dependent selection.

VI.ii. **Channel Sparsity & Concentration.** For each token x and layer ℓ, we compute the m channel contributions $c_j(x)$ and ask: how many channels carry most of the signal? We use two measures:

- 90% mass fraction: Sort $|c_j(x)|$ in descending order, cumulate, and find what fraction of the m channels is needed to account for 90% of the total $\sum_j |c_j(x)|$. If 10% of channels carry 90% of the mass, the decomposition is effectively sparse; if 90% of channels are needed, it is diffuse.

- Effective channel count: Treat $p_j = |c_j(x)|/\sum_k |c_k(x)|$ as a probability distribution and compute $\exp(H(p))$, the expected Shannon entropy. This equals m if all channels contribute equally and approaches 1 if a single channel dominates. It gives a smooth measure of the "effective number of active channels" per token.

The channel decomposition is moderately concentrated but not sparse (Figure 3). Across most layers, roughly 50–60% of channels are needed to capture 90% of the total channel magnitude, and the effective channel count is approximately 2500–3000 out of $m = 4864$. For context, sparse autoencoders in typical interpretability work have L_0 around 50–100 active features out of dictionaries of 16K–64K—orders of magnitude sparser.

Individual routed-CP channels $c_j(x)$ are therefore not naturally monosemantic. With $\sim$2500 channels simultaneously active per token, they are performing overlapping, distributed computation. The routing selects a moderately concentrated subset, but it is far from a sparse dictionary in the SAE sense.

The final layer is an exception. The 90% mass fraction drops to $\sim 45\%$ and the effective channel count falls sharply in layers 22–23. This is consistent with the bimodal α distribution observed in the routing statistics (Figure 2): many channels in the final layer are suppressed to near-zero α, so their contributions are negligible. The network performs sparser, more selective computation in the final FFN, which feeds directly into the unembedding and must support a sharp next-token prediction.

This result is relevant to the Tucker generalization argument. If the routed-CP channels were naturally very sparse—say, 50 channels active per token—one could argue that SwiGLU already provides a clean decomposition and there is less motivation for richer interaction structure. Instead, the same-index coupling constraint forces the network to spread its computation across thousands of channels, each representing a single rank-one interaction $w_j \otimes g_j$. A Tucker core that allows cross-channel interactions could in principle concentrate the same computation into fewer, more structured interaction modes.

VI.iii. **Routing Ablation**. This experiment isolates the contribution of input-dependent routing by replacing $\alpha_j(x)$ with constants at inference time and measuring the resulting perplexity. Mechanically, we swap $\mathrm{SiLU}(z) = zs(z)$ with $z \cdot C$ for some constant C, which turns each hidden unit into

$$h_j(x) = C(w_j^\top x)(g_j^\top x).$$

Three choices of C are tested:

- Uniform ($C = 0.5$): Near $z = 0$, $s(z) \approx 0.5$, so this preserves the correct scale for channels with small gate pre-activations.
- Per-channel mean ($C = \mathbb{E}_x[s(g_j^\top x)]$): This preserves the average routing strength for each channel, but removes all token-to-token variability.
- Ones ($C = 1$): Purely bilinear. No gating effect at all. This is the setting analyzed by [Pea+25] in their bilinear MLP work.

All three ablations are applied to every SwiGLU layer simultaneously. Perplexity is evaluated on a held-out 4096-token chunk of WikiText-2 that was not used for calibrating the per-channel means.

Routing is catastrophically important (Figure 4). The baseline model achieves a perplexity of 16.8. All three ablations destroy the model:

Ablation	Perplexity
Baseline (full routing)	16.8
Uniform ($\alpha = 0.5$)	272,375
Ones ($\alpha = 1$, bilinear)	737,654
Per-channel mean ($\alpha = \mathbb{E}[\alpha_j]$)	6,412,052

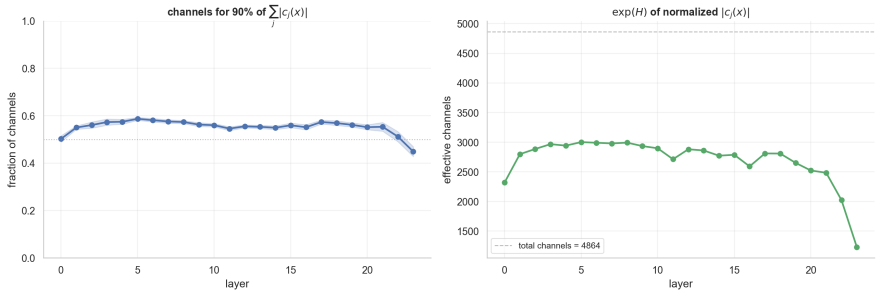


FIGURE 3. Channel sparsity across layers: 90% mass fraction (fraction of channels needed to capture 90% of total $|c_j|$) and effective channel count $\exp(H(p))$, by layer. The decomposition is moderately concentrated but far from sparse across most of the network, with a sharp drop in the final layers.

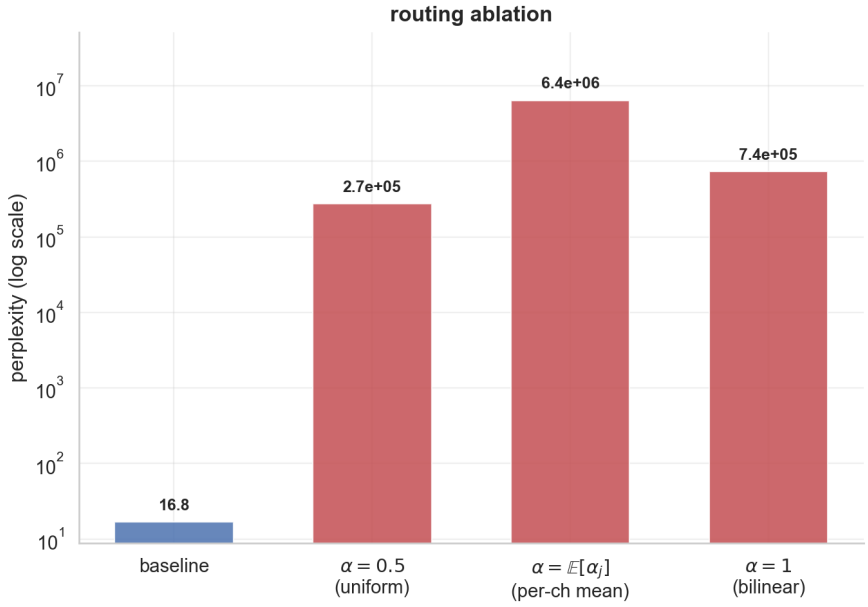


FIGURE 4. Perplexity under routing ablations. The baseline model achieves 16.8; all three constant-α replacements produce catastrophic degradation, confirming that input-dependent routing is structurally essential.

These are effectively random—the model has lost all useful computation under every ablation. The ordering among ablations is informative. Uniform (0.5) is the least catastrophic. This is consistent with the routing statistics: α distributions in early layers are centered near 0.5, so a constant of 0.5 at least approximates the right scale for most channels in most layers. Ones (1.0, bilinear) is worse because later layers have α distributions shifted well below 1. Replacing with 1.0 massively amplifies channels that should be partially suppressed, disrupting the learned activation scale.

Per-channel mean is worst, despite preserving the correct average scale per channel. This is the most revealing result. Consider a channel with mean $\alpha = 0.3$ and high variance: it is mostly off (α near 0), occasionally strongly on (α near 0.8–0.9). Replacing it with a constant 0.3 means it is always weakly on—never fully suppressed, never fully active. The tails of the routing distribution carry the input-specific information, and averaging them away is more destructive than a crude

uniform approximation that at least does not introduce systematic per-channel biases. The per-channel means are heterogeneous (the histograms show means ranging from ~ 0.1 to ~ 0.9), so this ablation simultaneously throttles channels that should occasionally fire strongly and keeps leaking signal through channels that should be mostly silent.

This does not appear to be a model that uses the sigmoid gate as a gentle fine-tuning mechanism on top of an approximately bilinear backbone. The input-dependent routing is structurally essential: the network's learned weight matrices W, G, U have been trained jointly with the sigmoid nonlinearity, and their magnitudes and directions are calibrated for the full nonlinear regime. Any constant-α surrogate produces outputs that are qualitatively wrong.

This directly validates a central premise of the routed-CP framework. The interaction tensor $\mathcal{A}(x) = \sum_j \alpha_j(x)\, u_j \otimes w_j \otimes g_j$ genuinely changes shape from token to token, and that shape change is what the model relies on. Analyzing only the bilinear skeleton (the [Pea+25] setting, equivalent to $\alpha \equiv 1$) discards the mechanism that makes the layer functional.

These experiments establish three empirical facts about how Qwen2.5-0.5B uses the routed-CP structure of its SwiGLU layers:

(1) **Routing is real and depth-dependent**. The routing coefficients $\alpha_j(x)$ are not approximately constant. Their variance increases with depth, and later layers develop bimodal routing distributions with a mix of always-on channels and selectively-gated channels. The network uses progressively sharper input-dependent selection in deeper layers.

(2) **The channel decomposition is moderately concentrated but not sparse**. Roughly 2500–3000 channels out of 4864 are effectively active per token. The routed-CP atoms are not individually monosemantic; there are too many of them active at once. This is consistent with our hypothesis that SwiGLU's same-index coupling constraint forces computation to be spread across many channels, each limited to a single rank-one interaction.

(3) **Removing input-dependent routing destroys the model**. None of our tested constant replacements for $\alpha_j(x)$ preserves useful model behavior. The interaction tensor must change shape from token to token. The bilinear case $\alpha = 1$ studied by [Pea+25] is not a useful approximation of the full SwiGLU computation.

Together, these results confirm that the routed-CP decomposition identifies structure that the model actively exploits. They also motivate the Tucker generalization: if input-dependent routing is essential but the current channel decomposition is too diffuse to be directly interpretable, then learning a cross-channel interaction core that concentrates computation into fewer modes is a natural next step.

References

[Vas+17] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. *Attention Is All You Need*. Advances in Neural Information Processing Systems, 2017.

[Sha20] N. Shazeer. *GLU Variants Improve Transformer*. arXiv:2002.05202, 2020.

[KB09] T. G. Kolda and B. W. Bader. *Tensor Decompositions and Applications*. SIAM Review, **51**(3):455–500, 2009.

[Nad64] E. A. Nadaraya. *On Estimating Regression*. Theory of Probability and its Applications, **9**(1):141–142, 1964.

[Pai25] D. Pai. *Regression is All You Need*. Tilde Research, 2025. https://blog.tilderesearch.com/vignettes/regression

[JSSB20] S. M. Jayakumar, J. Menick, W. M. Czarnecki, J. Schwarz, J. Rae, S. Osindero, Y. W. Teh, T. Harley, and R. Pascanu. *Multiplicative Interactions and Where to Find Them*. ICLR, 2020.

[Ols+22] C. Olsson, N. Elhage, N. Nanda, N. Joseph, N. DasSarma, T. Henighan, et al. *In-context Learning and Induction Heads*. Transformer Circuits Thread, 2022.

[Fu+23] D. Y. Fu, T. Dao, K. K. Saab, A. W. Thomas, A. Rudra, and C. Ré. *Hungry Hungry Hippos: Towards Language Modeling with State Space Models*. ICLR, 2023.

[PKW24] D. Pai, A. Keigwin, and A. Wang. *The Rate Distortion Dance of Sparse Autoencoders*. Tilde Research, 2024. https://blog.tilderesearch.com/blog/rate-distortion-saes

[Bri+23] T. Bricken, A. Templeton, J. Batson, B. Chen, A. Jermyn, T. Conerly, et al. *Towards Monosemanticity: Decomposing Language Models with Dictionary Learning*. Transformer Circuits Thread, 2023.

[Pea+25] M. T. Pearce, T. Dooms, A. Rigg, J. M. Oramas, and L. Sharkey. *Bilinear MLPs Enable Weight-Based Mechanistic Interpretability*. arXiv:2410.08417, 2025.